ANALIZADOR DE PSEUDOCODIGO

MANUAL TECNICO

Presentación

El siguiente manual se ha desarrollado con la finalidad de dar a conocer la información necesaria para realizar mantenimiento, instalación y exploración del software, el cual consta de diferentes funcionalidades. El manual ofrece la información necesaria de ¿Cómo está realizado el software? Para que la persona (Desarrollador/Ingeniero) que quiera editar el software lo haga de una manera apropiada, dando a conocer la estructura del desarrollo del aplicativo. La siguiente guía se encuentra dividida en las herramientas que se usaron para la creación del software con una breve explicación paso a paso.

Introducción

El manual se realiza con el fin de detallar el software para el Analizador de Pseudocodigo en términos técnicos para que la persona que vaya a administrar, editar o configurar el aplicativo lo haga de una manera apropiada. El documento se encuentra dividido en las siguientes secciones:

- ASPECTOS TEORICOS: Se dará a conocer conceptos, definiciones y explicaciones de los componentes del aplicativo desde un punto de vista teórico para mayor entendimiento por parte del lector sobre el funcionamiento del sistema de información y herramientas.
- DIAGRAMAS DE MODELAMIENTO: Se compone por diagramas e ilustraciones alusivas al funcionamiento del aplicativo (Los diagramas se tienen en archivos diferentes al actual, buscar las referencias).

ASPECTOS TECNICOS

El aplicativo tiene como finalidad interpretar pseudocodigo a su respectivo diagrama de flujo. Se recomienda que el siguiente manual sea manipulado únicamente por la persona que quiera administrar, editar o configurar el software para velar por la seguridad de los daos que se almacenan.

1. HERRAMINETAS UTILIZADAS: En esta sección se procede a explicar las herramientas informáticas empleadas para el desarrollo del aplicativo:
    a. Android Studio: Android Studio es el entorno de desarrollo integrado (IDE) oficial creado por Google para diseñar, codificar, probar y depurar aplicaciones para la plataforma Android. Basado en IntelliJ IDEA, proporciona herramientas como un emulador rápido, sistema de compilación Gradle y soporte para Kotlin, Java y C++. Para la descarga e

Instalación del IDE puede ir a su pagina oficial y seguir la guía de instalación https://developer.android.com/studio/install?hl=es-419

b. JFlex: Es un generador de analizadores léxicos (escáneres) de código abierto, rápido y eficiente, diseñado específicamente para Java. Toma un archivo de especificación con expresiones regulares y genera código Java que clasifica el texto de entrada en tokens (palabras clave, operadores, etc.), siendo fundamental en la creación de compiladores e intérpretes. Para la descarga e Instalación de la herramienta puede ir a su página oficial y descargar el archivo comprimido, para posteriormente extraer el archivo .jar https://www.jflex.de/download.html

c. Java CUP: Es una herramienta diseñada para la creación de analizadores sintácticos (parsers) en el lenguaje de programación Java. Utiliza una gramática LALR(1) para generar código Java que puede reconocer estructuras de lenguaje, siendo frecuentemente usado junto con JFlex para el análisis léxico. Para la Descarga e Instalación de la herramienta puede ir a su pagina oficial y descargar el archivo comprimido, para posteriormente extraer los archivos .jar https://www2.cs.tum.edu/projects/cup/install.php

2. ESTRUCTURA DEL PROYECTO: En esta sección se procederá a explicar las clases/archivos que tienen una relevancia significativa en el funcionamiento del software:

a. Analyzer.kt: Esta clase es la encargada de interactuar con las clases Lexer y Parser (generadas por JFlex y CUP respectivamente), para obtener los resultados del análisis al pseudocodigo ingresado.

b. Lexer.java: Clase generada por JFlex, la cual contiene el Autómata Finito Determinista para el análisis léxico del seudocódigo de entrada. Esta clase se genera en base al archivo de configuración especificado para el análisis léxico.

c. Parser.java: Clase generada por CUP, la cual contiene el código necesario para el análisis sintáctico del pseudocodigo de entrada. Esta clase se genera en base al archivo de configuración especificado para el análisis sintáctico.

d. Lexer.flex: Archivo de configuración para que JFlex genere una Clase Java que hará el análisis léxico en base al archivo de configuración.

e. Parser.cup: Archivo de configuración para que CUP genere una Clase Java que hará el análisis sintáctico en base al archivo de configuración.

f. Component.kt: Clase que será la base para definir cada tipo de objeto/forma dentro del diagrama de flujo.

g. Bloque.kt, Fin.kt, Inicio.kt: Clases que heredan de Component implementando atributos específicos para representar específicamente al objeto/forma dentro del diagrama de flujo.
h. Si.kt y Mientras.kt: Clases que heredan de Component implementando atributos especifico y además añadiendo un atributo de la clase Bloque que representara al bloque de código que se encuentra adentro de la estructura de control.
i. CodeType.kt, FigureType.kt y FontType.kt: Son enums que contienen las constantes de los tipos de códigos/estructuras que se manejan en el pseudocodigo (CodeType), contienen las constantes de los tipos de figuras que tendrá el diagrama de flujo (FigureType) y contienen los tipos de fuente que tendrá el diagrama de flujo.
j. ErrorReport.ky, OperatorReport.kt y StructureReport.kt: Son clases que modelan los datos de los reportes que se van a representar como parte del resultado del análisis al pseudocodigo.
k. Interpreter.kt: Clase que hace las modificaciones a nivel de estilo a los componentes/figuras que se van a representar en el diagrama de flujo y genera el código DOT que servirá para generar el diagrama de flujo.
l. ParserUtilities.kt: Clase que ayuda al Parser a llevar el control de los IDs de cada componente/figura que se va a representar, y también ayuda a generar los bloques de código que van dentro de las estructuras de control.
m. TableActivity.kt: Clase que genera/construye las tablas de reportes en base a las listas de objetos de reporte que obtiene del análisis realizado al pseudocodigo.
n. MainActivity.kt: Clase principal que controla el flujo de navegación y presentación de la UI.

3. GRAMATICA PARA ANALIZADOR LEXICO: En esta sección se procede a explicar parte esencial del archivo de configuración .flex para generar el análisis léxico dentro de la aplicación, es decir, se va a explicar las expresiones regulares utilizadas para el análisis:
   a. Constantes:

```
// REGULAR EXPRESSIONS
LineTerminator = \r|\n|\n\r|\r\n
WhiteSpace = [ \t\f]
InputCharacter = [^\r\n]
Dot = \.
WholeNumber = 0|[1-9][0-9]*
DecimalNumber = {WholeNumber}{Dot}[0-9]+
Letter = [a-zA-Z]
ID = _?{Letter}({Letter}|_|{WholeNumber})*
```

b. Palabras Reservadas:

```
"%%%%"                  { return symbol(sym.SEPARADOR); }
"INICIO"                { return symbol(sym.INICIO, yytext()); }
"FIN"                   { return symbol(sym.FIN, yytext()); }

"SI"                    { return symbol(sym.SI); }
"ENTONCES"              { return symbol(sym.ENTONCES); }

"MIENTRAS"              { return symbol(sym.MIENTRAS); }
"HACER"                 { return symbol(sym.HACER); }

"VAR"                   { return symbol(sym.VAR, yytext()); }
"MOSTRAR"               { return symbol(sym.MOSTRAR, yytext()); }
"LEER"                  { return symbol(sym.LEER, yytext()); }

"ELIPSE"                { return symbol(sym.ELIPSE); }
"CIRCULO"               { return symbol(sym.CIRCULO); }
"PARALELOGRAMO"         { return symbol(sym.PARALELOGRAMO); }
"RECTANGULO"            { return symbol(sym.RECTANGULO); }
"ROMBO"                 { return symbol(sym.ROMBO); }
"RECTANGULO_REDONDEADO" { return symbol(sym.RECTANGULO_REDONDEADO); }

"ARIAL"                 { return symbol(sym.ARIAL); }
"TIMES_NEW_ROMAN"        { return symbol(sym.TIMES_NEW_ROMAN); }
"COMIC_SANS"            { return symbol(sym.COMIC_SANS); }
"VERDANA"               { return symbol(sym.VERDANA); }
```

```
"%DEFAULT" { return symbol(sym.DEFAULT); }

"%COLOR_TEXTO_SI" { return symbol(sym.COLOR_TEXTO_SI); }
"%COLOR_SI"       { return symbol(sym.COLOR_SI); }
"%FIGURA_SI"      { return symbol(sym.FIGURA_SI); }
"%LETRA_SI"       { return symbol(sym.LETRA_SI); }
"%LETRA_SIZE_SI"  { return symbol(sym.LETRA_SIZE_SI); }

"%COLOR_TEXTO_MIENTRAS" { return symbol(sym.COLOR_TEXTO_MIENTRAS); }
"%COLOR_MIENTRAS"       { return symbol(sym.COLOR_MIENTRAS); }
"%FIGURA_MIENTRAS"      { return symbol(sym.FIGURA_MIENTRAS); }
"%LETRA_MIENTRAS"       { return symbol(sym.LETRA_MIENTRAS); }
"%LETRA_SIZE_MIENTRAS"  { return symbol(sym.LETRA_SIZE_MIENTRAS); }

"%COLOR_TEXTO_BLOQUE" { return symbol(sym.COLOR_TEXTO_BLOQUE); }
"%COLOR_BLOQUE"       { return symbol(sym.COLOR_BLOQUE); }
"%FIGURA_BLOQUE"      { return symbol(sym.FIGURA_BLOQUE); }
"%LETRA_BLOQUE"       { return symbol(sym.LETRA_BLOQUE); }
"%LETRA_SIZE_BLOQUE"  { return symbol(sym.LETRA_SIZE_BLOQUE); }
```

c. Operadores Relacionales:

```
"==" { return symbol(sym.IGUALDAD, yytext()); }
"!=" { return symbol(sym.DIFERENTE, yytext()); }
">"  { return symbol(sym.MAYOR, yytext()); }
"<"  { return symbol(sym.MENOR, yytext()); }
">=" { return symbol(sym.MAYOR_IGUAL, yytext()); }
"<=" { return symbol(sym.MENOR_IGUAL, yytext()); }
```

d. Operadores Lógicos:

```
"&&" { return symbol(sym.AND, yytext()); }
"||" { return symbol(sym.OR, yytext()); }
"!"  { return symbol(sym.NOT, yytext()); }
```

e. Operadores Matemáticos:

```
"+" { return symbol(sym.SUMA); }
"-" { return symbol(sym.RESTA); }
"*" { return symbol(sym.MULTIPLICACION); }
"/" { return symbol(sym.DIVISION); }
"(" { return symbol(sym.PARENTESIS_ABIERTO, yytext()); }
")" { return symbol(sym.PARENTESIS_CERRADO, yytext()); }
```

f. Colores Hexadecimales:

```
"H"[a-fA-F0-9]{6} {
    this.string.setLength(0);
    this.string.append(yytext());
    this.string.deleteCharAt(0);
    return symbol(sym.HEXADECIMAL, this.string.toString().trim());
}
"H"[a-fA-F0-9]{3} {
    this.string.setLength(0);
    this.string.append(yytext());
    this.string.deleteCharAt(0);
    return symbol(sym.HEXADECIMAL, this.string.toString().trim());
}
```

g. Expresiones Regulares con Estados:

```
"#" { this.string.setLength(0); yybegin(COMMENT); }

\" { System.out.println("SE EMPIEZA TEXTO"); this.string.setLength(0); yybegin(TEXT); }
```

```
// Comment state
<COMMENT> {
    {InputCharacter}+ { this.string.append(yytext()); }
    {LineTerminator} {
        yybegin(YYINITIAL);
        //return symbol(sym.COMENTARIO, this.string.toString().trim());
    }
    <<EOF>> {
        yybegin(YYINITIAL);
        //return symbol(sym.COMENTARIO, this.string.toString().trim());
    }
}

// Text state
<TEXT> {
    \" {
        System.out.println("TEXTO:" + this.string.toString().trim());
        yybegin(YYINITIAL);
        return symbol(sym.TEXTO, this.string.toString().trim());
    }
    {InputCharacter} { this.string.append(yytext()); }
    {LineTerminator} { this.string.append("\n"); }
}
```

h. Expresiones Regulares Útiles:

```
{WholeNumber}   { return symbol(sym.NUMERO_ENTERO, Integer.parseInt(yytext())); }
{DecimalNumber} { return symbol(sym.NUMERO_DECIMAL, Double.parseDouble(yytext())); }
{ID}            { return symbol(sym.ID, yytext()); }
```

```
/* Ignored whitespace */
{WhiteSpace} | {LineTerminator} { /* Ignore */ }

/* Error handling */
. { error(yytext()); }
<<EOF>> { return symbol(sym.EOF); }
```

4. GRAMATICA PARA ANALIZADOR SINTACTICO: En esta sección se procede a explicar parte esencial del archivo de configuración .cup para generar el análisis sintáctico dentro de la aplicación, es decir, se va a explicar las gramáticas utilizadas para el análisis: (Nota: Los símbolos Terminales están en Mayúscula y los símbolos No Terminales están en Minúscula)

```
start with fuente_entrada;

fuente_entrada ::= parte1 SEPARADOR contenido_configuracion;

parte1 ::= INICIO contenido_codigo FIN;

contenido_codigo ::= contenido_codigo instruccion_codigo |
                     instruccion_codigo

instruccion_codigo ::= si |
                       mientras |
                       instruccion_simple

contenido_bloque ::= lista_instrucciones;

lista_instrucciones ::= lista_instrucciones instruccion_simple |
                        instruccion_simple;

instruccion_simple ::= variable |
            mostrar |
            leer

variable ::= VAR ID asignacion |
             ID IGUAL operacion_matematica_literal;

asignacion ::= IGUAL operacion_matematica_literal |
               /* empty */;

si ::= SI PARENTESIS_ABIERTO operacion_lra PARENTESIS_CERRADO
ENTONCES contenido_bloque_opt FIN SI

contenido_bloque_opt ::= contenido_bloque |
                         /* empty */
operacion_lra ::= operacion_lra OR operacion_and |
                  operacion_and;
```

```
operacion_and ::= operacion_and AND operacion_not |
                  operacion_not;

operacion_not ::= NOT operacion_lra |
                  operacion_relacional;

operacion_relacional ::= operacion_matematica_literal operador_relacional
operacion_matematica_literal |
                         operacion_matematica_literal;

operacion_matematica_literal ::= operacion_matematica_literal operador_sr tl |
                                 tl;

tl ::= tl operador_md fl |
       fl;

fl ::= PARENTESIS_ABIERTO operacion_lra PARENTESIS_CERRADO |
     NUMERO_DECIMAL |
     NUMERO_ENTERO |
     ID;

operacion_matematica_numerica ::= operacion_matematica_numerica
operador_sr tn |
                                 tn;

tn ::= tn operador_md fn |
       fn;

fn ::= PARENTESIS_ABIERTO operacion_matematica_numerica
PARENTESIS_CERRADO |
       NUMERO_DECIMAL |
       NUMERO_ENTERO;

mientras ::= MIENTRAS PARENTESIS_ABIERTO operacion_lra
PARENTESIS_CERRADO HACER contenido_bloque_opt FIN MIENTRAS;

mostrar ::= MOSTRAR mensaje;

mensaje ::= TEXTO |
```

```
        ID;

leer ::= LEER ID;

contenido_configuracion ::= contenido_configuracion instruccion_configuracion |
                            instruccion_configuracion;

instruccion_configuracion ::= instruccion_default |
                              instruccion_color_texto_si |
                              instruccion_color_si |
                              instruccion_figura_si |
                              instruccion_letra_si |
                              instruccion_letra_size_si |
                              instruccion_color_texto_mientras |
                              instruccion_color_mientras |
                              instruccion_figura_mientras |
                              instruccion_letra_mientras |
                              instruccion_letra_size_mientras |
                              instruccion_color_texto_bloque |
                              instruccion_color_bloque |
                              instruccion_figura_bloque |
                              instruccion_letra_bloque |
                              instruccion_letra_size_bloque;

instruccion_default ::= DEFAULT IGUAL NUMERO_ENTERO;

instruccion_color_texto_si ::= COLOR_TEXTO_SI IGUAL color PIPE
NUMERO_ENTERO;
instruccion_color_si ::= COLOR_SI IGUAL color PIPE NUMERO_ENTERO;
instruccion_figura_si ::= FIGURA_SI IGUAL nombre_figura PIPE
NUMERO_ENTERO;
instruccion_letra_si ::= LETRA_SI IGUAL nombre_letra PIPE
NUMERO_ENTERO;
instruccion_letra_size_si ::= LETRA_SIZE_SI IGUAL
operacion_matematica_numerica PIPE NUMERO_ENTERO;

instruccion_color_texto_mientras ::= COLOR_TEXTO_MIENTRAS IGUAL color
PIPE NUMERO_ENTERO;
instruccion_color_mientras ::= COLOR_MIENTRAS IGUAL color PIPE
NUMERO_ENTERO;
```

```
instruccion_figura_mientras ::= FIGURA_MIENTRAS IGUAL nombre_figura PIPE
NUMERO_ENTERO;
instruccion_letra_mientras ::= LETRA_MIENTRAS IGUAL nombre_letra PIPE
NUMERO_ENTERO;
instruccion_letra_size_mientras ::= LETRA_SIZE_MIENTRAS IGUAL
operacion_matematica_numerica PIPE NUMERO_ENTERO;

instruccion_color_texto_bloque ::= COLOR_TEXTO_BLOQUE IGUAL color PIPE
NUMERO_ENTERO;
instruccion_color_bloque ::= COLOR_BLOQUE IGUAL color PIPE
NUMERO_ENTERO;
instruccion_figura_bloque ::= FIGURA_BLOQUE IGUAL nombre_figura PIPE
NUMERO_ENTERO;
instruccion_letra_bloque ::= LETRA_BLOQUE IGUAL nombre_letra PIPE
NUMERO_ENTERO;
instruccion_letra_size_bloque ::= LETRA_SIZE_BLOQUE IGUAL
operacion_matematica_numerica PIPE NUMERO_ENTERO;

color ::= rgb |
        HEXADECIMAL;

rgb ::= operacion_matematica_numerica COMA
operacion_matematica_numerica COMA operacion_matematica_numerica;

operador_relacional ::= IGUALDAD |
                    DIFERENTE |
                    MAYOR |
                    MENOR |
                    MAYOR_IGUAL |
                    MENOR_IGUAL;

operador_sr ::= SUMA |
            RESTA;

operador_md ::= MULTIPLICACION |
            DIVISION;

nombre_figura ::= ELIPSE |
               CIRCULO |
               PARALELOGRAMO |
```

```
                RECTANGULO |
                ROMBO |
                RECTANGULO_REDONDEADO;

nombre_letra ::= ARIAL |
               TIMES_NEW_ROMAN |
               COMIC_SANS |
               VERDANA;
```